

Diseño de Clases UML: DataForge IA

1. Introducción

Este documento presenta el diseño detallado de clases UML para el sistema **DataForge IA**. El diagrama de clases es una representación estática de la estructura del sistema, mostrando las clases principales, sus atributos, métodos y las relaciones entre ellas. Este modelo sirve como plano fundamental para la implementación orientada a objetos, asegurando que la lógica de negocio y la gestión de datos estén correctamente encapsuladas y relacionadas.

2. Clases Principales, Atributos y Métodos

El modelo se compone de entidades que representan los conceptos clave del dominio del problema. A continuación, se detallan las clases principales:

2.1. Gestión de Usuarios y Cuentas

- **Cuenta** : Representa la entidad organizativa (la PYME) que suscribe el servicio.
 - *Atributos*: `id` (UUID), `nombreEmpresa` (String), `planSuscripcion` (Enum: Básico, Pro, Enterprise), `fechaCreacion` (Date), `estado` (Enum: Activa, Suspendida).
 - *Métodos*: `actualizarPlan(nuevoPlan)` , `suspenderCuenta()` , `obtenerUsoAlmacenamiento()` .
- **Usuario (Clase Abstracta)**: Define las propiedades comunes de cualquier usuario del sistema.
 - *Atributos*: `id` (UUID), `nombre` (String), `email` (String), `passwordHash` (String), `fechaRegistro` (Date), `ultimoAcceso` (Date).
 - *Métodos*: `autenticar(password): Boolean` , `actualizarPerfil(datos)` , `restablecerPassword()` .
- **Administrador (Hereda de Usuario)**: Usuario con privilegios para gestionar la cuenta de la empresa.
 - *Atributos*: `nivelAcceso` (Integer).
 - *Métodos*: `invitarUsuario(email, rol)` , `eliminarUsuario(idUsuario)` , `gestionarFacturacion()` .
- **Analista (Hereda de Usuario)**: Usuario estándar que interactúa con los datos y modelos.
 - *Atributos*: `departamento` (String).

- *Métodos:* `crearConjuntoDatos()` , `configurarModelo()` , `generarReporte()` .

2.2. Gestión de Datos

- **ConjuntoDeDatos** : Representa un archivo de datos cargado por un usuario.
 - *Atributos:* `id` (UUID), `nombre` (String), `formato` (Enum: CSV, XLSX), `tamañoBytes` (Long), `rutaAlmacenamiento` (String), `fechaCarga` (Date), `estadoProcesamiento` (Enum: Pendiente, Procesado, Error).
 - *Métodos:* `cargarArchivo(archivo)` , `validarFormato(): Boolean` , `eliminarDatos()` .
- **EsquemaDatos** : Define la estructura y el mapeo de las columnas de un `ConjuntoDeDatos` .
 - *Atributos:* `id` (UUID), `columnas` (List).
 - *Métodos:* `definirMapeo(mapeo)` , `validarTipos()` .
- **Columna** : Representa una columna individual dentro de un esquema.
 - *Atributos:* `nombreOriginal` (String), `nombreMapeado` (String), `tipoDato` (Enum: Numerico, Categórico, Fecha, Texto), `esObjetivo` (Boolean).

2.3. Modelado Predictivo

- **ModeloPredictivo (Clase Abstracta)**: Define la interfaz común para todos los algoritmos de IA.
 - *Atributos:* `id` (UUID), `nombre` (String), `descripcion` (String), `estado` (Enum: NoEntrenado, Entrenando, Listo, Error), `fechaUltimoEntrenamiento` (Date).
 - *Métodos:* `configurarParametros(parametros)` , `entrenar(datos: ConjuntoDeDatos)` , `predecir(datosEntrada): Prediccion` .
- **ModeloVentas (Hereda de ModeloPredictivo)**: Implementación específica para predicción de ventas (ej. Regresión Lineal, ARIMA).
 - *Atributos:* `horizonteTiempo` (Integer), `frecuencia` (Enum: Diario, Semanal, Mensual).
- **ModeloInventario (Hereda de ModeloPredictivo)**: Implementación específica para predicción de demanda de inventario.
 - *Atributos:* `nivelConfianza` (Float), `leadTime` (Integer).

2.4. Resultados y Visualización

- **Prediccion** : Encapsula los resultados generados por un modelo.

- *Atributos*: `id` (UUID), `fechaGeneracion` (Date), `resultadosRaw` (JSON/Map), `metricasPrecision` (Map).
- *Métodos*: `exportarCSV(): Archivo`, `obtenerResumen(): String`.
- **Reporte** : Representa una visualización estructurada de las predicciones.
 - *Atributos*: `id` (UUID), `titulo` (String), `tipoGrafico` (Enum: Lineas, Barras, Pastel), `configuracionVisual` (JSON).
 - *Métodos*: `generarPDF(): Archivo`, `actualizarDatos(nuevaPrediccion)`.
- **Alerta** : Define una condición sobre una predicción que dispara una notificación.
 - *Atributos*: `id` (UUID), `metricaObjetivo` (String), `condicion` (Enum: MayorQue, MenorQue, Igual), `umbral` (Float), `activa` (Boolean).
 - *Métodos*: `evaluarCondicion(prediccion): Boolean`, `dispararNotificacion()`.

3. Relaciones entre Clases

El modelo establece las siguientes relaciones estructurales:

- **Composición**:
 - Una `Cuenta` está compuesta por uno o más `Usuario`s. Si la cuenta se elimina, los usuarios asociados también deberían ser gestionados (eliminados o archivados).
 - Un `ConjuntoDeDatos` está compuesto por un único `EsquemaDatos`.
 - Un `EsquemaDatos` está compuesto por una o más `Columna`s.
- **Asociación**:
 - Un `Usuario` (específicamente un `Analista`) *gestiona* cero o más `ConjuntoDeDatos`.
 - Un `ConjuntoDeDatos` *es utilizado por* cero o más `ModeloPredictivo`s para su entrenamiento.
 - Un `ModeloPredictivo` *genera* cero o más `Prediccion`es.
 - Una `Prediccion` *es visualizada en* cero o más `Reporte`s.
 - Una `Prediccion` *es monitoreada por* cero o más `Alerta`s.
 - Un `Usuario` *crea* cero o más `Reporte`s y `Alerta`s.
- **Herencia (Generalización)**:
 - `Administrador` y `Analista` heredan de la clase abstracta `Usuario`.

- `ModeloVentas` y `ModeloInventario` heredan de la clase abstracta `ModeloPredictivo`.

4. Explicación del Modelo

El diseño de clases propuesto para DataForge IA sigue los principios de la programación orientada a objetos (POO), garantizando un alto grado de cohesión y un bajo acoplamiento.

- 1. Aislamiento de Responsabilidades:** La separación entre la gestión de usuarios (`Cuenta` , `Usuario`), la gestión de datos (`ConjuntoDeDatos` , `EsquemaDatos`) y la lógica de IA (`ModeloPredictivo` , `Prediccion`) permite que cada dominio evolucione de forma independiente. Por ejemplo, se pueden añadir nuevos tipos de modelos predictivos sin alterar la lógica de carga de datos.
- 2. Uso de Herencia para Extensibilidad:** La clase abstracta `ModeloPredictivo` define un contrato estándar (`entrenar` , `predecir`). Las clases concretas (`ModeloVentas` , `ModeloInventario`) implementan los algoritmos específicos. Esto facilita la incorporación futura de nuevos modelos (ej. `ModeloRotacionClientes`) utilizando el patrón de diseño *Strategy* o *Factory*.
- 3. Flexibilidad en los Datos:** La separación entre `ConjuntoDeDatos` (el archivo físico/lógico) y `EsquemaDatos` (la interpretación de las columnas) permite que el sistema maneje archivos con estructuras variables, adaptándose a las diferentes formas en que las PYMES exportan su información.
- 4. Trazabilidad:** Las relaciones aseguran que siempre se pueda rastrear qué usuario cargó qué datos, qué modelo se entrenó con esos datos y qué reportes se generaron a partir de las predicciones resultantes. Esto es crucial para la auditoría y la reproducibilidad de los análisis.

5. Código PlantUML del Diagrama de Clases

```
@startuml
skinparam classAttributeIconSize 0

class Cuenta {
    - id: UUID
    - nombreEmpresa: String
    - planSuscripcion: PlanEnum
    - fechaCreacion: Date
    - estado: EstadoCuentaEnum
    + actualizarPlan(nuevoPlan)
    + suspenderCuenta()
    + obtenerUsoAlmacenamiento(): Long
}

abstract class Usuario {
    - id: UUID
    - nombre: String
    - email: String
    - passwordHash: String
    - fechaRegistro: Date
    - ultimoAcceso: Date
    + autenticar(password: String): Boolean
    + actualizarPerfil(datos: Map)
    + restablecerPassword()
}

class Administrador {
    - nivelAcceso: Integer
    + invitarUsuario(email: String, rol: String)
    + eliminarUsuario(idUsuario: UUID)
    + gestionarFacturacion()
}

class Analista {
    - departamento: String
    + crearConjuntoDatos()
    + configurarModelo()
    + generarReporte()
}
```

```
class ConjuntoDeDatos {
    - id: UUID
    - nombre: String
    - formato: FormatoEnum
    - tamañoBytes: Long
    - rutaAlmacenamiento: String
    - fechaCarga: Date
    - estadoProcesamiento: EstadoDatosEnum
    + cargarArchivo(archivo: File)
    + validarFormato(): Boolean
    + eliminarDatos()
}

class EsquemaDatos {
    - id: UUID
    + definirMapeo(mapeo: Map)
    + validarTipos(): Boolean
}

class Columna {
    - nombreOriginal: String
    - nombreMapeado: String
    - tipoDato: TipoDatoEnum
    - esObjetivo: Boolean
}

abstract class ModeloPredictivo {
    - id: UUID
    - nombre: String
    - descripcion: String
    - estado: EstadoModeloEnum
    - fechaUltimoEntrenamiento: Date
    + configurarParametros(parametros: Map)
    + entrenar(datos: ConjuntoDeDatos)
    + predecir(datosEntrada: Map): Prediccion
}

class ModeloVentas {
    - horizonteTiempo: Integer
    - frecuencia: FrecuenciaEnum
}
```

```

class ModeloInventario {
    - nivelConfianza: Float
    - leadTime: Integer
}

class Prediccion {
    - id: UUID
    - fechaGeneracion: Date
    - resultadosRaw: JSON
    - metricasPrecision: Map<String, Float>
    + exportarCSV(): File
    + obtenerResumen(): String
}

class Reporte {
    - id: UUID
    - titulo: String
    - tipoGrafico: TipoGraficoEnum
    - configuracionVisual: JSON
    + generarPDF(): File
    + actualizarDatos(nuevaPrediccion: Prediccion)
}

class Alerta {
    - id: UUID
    - metricaObjetivo: String
    - condicion: CondicionEnum
    - umbral: Float
    - activa: Boolean
    + evaluarCondicion(prediccion: Prediccion): Boolean
    + dispararNotificacion()
}

Usuario <-- Administrador
Usuario <-- Analista

Cuenta *-- "1..*" Usuario : contiene >

Analista "1" -- "0..*" ConjuntoDeDatos : gestiona >
ConjuntoDeDatos *-- "1" EsquemaDatos : tiene >
EsquemaDatos *-- "1..*" Columna : define >

```

```
ConjuntoDeDatos "1" -- "0..*" ModeloPredictivo : entrena >  
ModeloPredictivo <-- ModeloVentas  
ModeloPredictivo <-- ModeloInventario
```

```
ModeloPredictivo "1" -- "0..*" Prediccion : genera >  
Prediccion "1" -- "0..*" Reporte : visualizada en >  
Prediccion "1" -- "0..*" Alerta : monitoreada por >
```

```
Analista "1" -- "0..*" Reporte : crea >  
Analista "1" -- "0..*" Alerta : configura >
```

```
@enduml
```